

Exercices sur la vérification déductive

Enseignement à distance de l'Université Marie et Louis Pasteur
Spécification et preuve de programmes

Version du 7 novembre 2025

Objectifs

- Comprendre les enjeux de la tâche de vérification des logiciels : nécessité de vérifier, démarche de vérification, complexité de la vérification.
- Comprendre naïvement les solutions de test et de test exhaustif.
- Faire la différence entre test et vérification.

1 Exercice

L'exercice porte sur le programme C de la figure 1, qui implémente un algorithme de recherche dichotomique de la position d'un entier x dans un tableau t de n entiers, trié dans l'ordre croissant. L'intervalle des entiers entre n et m inclus est noté $n..m$. Les indices du tableau t valides sont dans l'intervalle $0..n-1$.

```
1 int binary_search(int t[], int n, int x) {  
2   int L = -1, R = n-1, m;  
3  
4   while(L  $\neq$  R) {  
5     m = (L+R)/2;  
6     if (t[m] > x)  
7       R = m-1;  
8     else  
9       L = m;  
10  }  
11  return L;  
12 }
```

FIGURE 1 – Recherche dichotomique dans un tableau trié

1. Quel résultat ce programme va-t-il retourner pour $n = 5$, $x = 5$ et le tableau t suivant ?

i	0	1	2	3	4
$t[i]$	2	4	7	9	9

2. Quel résultat ce programme va-t-il retourner pour $n = 5$, $x = 7$ et le même tableau t ?
3. A votre avis cet algorithme est-il juste ? Pourquoi ?
4. Exécuter le programme pour $n = 5$ et le même tableau t , mais pour $x=9$. Le programme est-il correct ? Si ce n'est pas le cas, quel(s) défaut(s) avez-vous trouvé ?

5. Proposer une version correcte de ce programme.
6. Si un tel programme était intégré au système de freinage ABS de votre voiture, auriez-vous confiance en sa vérification à partir de quelques tests ? Si non, comment vous y prendriez-vous pour le vérifier ?
7. Si nous faisons la vérification par un test exhaustif pour le cas $n=5$ avec x quelconque et t quelconque, en supposant qu'un entier peut prendre 65536 valeurs différentes (entier 16 bits) et qu'une exécution prend 1 seconde, quel serait le temps nécessaire pour effectuer ce test exhaustif ?
8. Puisque le test exhaustif n'est pas possible, comment vérifier si ce programme est juste (ou correct) ?
9. Mais qu'est-ce qu'un programme "correct" ? De quoi doit-on disposer pour pouvoir parler précisément de correction d'un programme ? Est-ce que correction est synonyme de "conformité" ?
10. Quel bilan peut-on faire concernant la vérification de programmes à partir de cet exercice ?